نشریهٔ انجمن کدنویسی

مسیر ۱۲ درسی Rust تمام شد؛ حالا نوبت توست بسازی!

🦀 پروندهٔ ویژه: Rust از صفر تا سازه‌ها

در این شماره: کامل‌شدن ۱۲ درس Rust · درسنامهٔ توابع با return · چالش برنامه‌نویسی آبان · آدا لولیس، اولین برنامه‌نویس تاریخ · اپ‌های تازه · بازی کدِ رمزی

```
fn main() {
    println!("Salam, Aban 1405!");
}
```

سرمقاله

آبان امسال برای انجمن کدنویسی ماهِ بردها بود. درست همان‌طور که قول داده بودیم، مسیر «آموزش Rust از صفر» با درس یازدهم و دوازدهم به پایان رسید و الان، هر عضو انجمن که هر دوازده درس را خوانده باشد، از «Rust چیست؟» تا «سازه‌ها» راه را کامل رفته است. این دوازده درس سه ماه دوید؛ سریع‌تر از تصورمان.

اما تمام‌شدن درس‌ها پایان راه نیست؛ شروع است. زبان فقط ابزار است و ابزار وقتی ارزش دارد که چیزی با آن ساخته شود. برای همین چالش آبان را طراحی کردیم: با همان آموخته‌های دوازده درس، یک پروژهٔ کوچک واقعی بساز و در کانال به اشتراک بگذار. جزئیاتش در صفحهٔ چالش این شماره است.

خبرنامهٔ انجمن

اتفاق
انتشار درس ۱۱ — توابع با return؛ دو درس مانده به پایان مسیر
انتشار درس ۱۲ — سازه‌ها و کامل‌شدن مسیر ۱۲ درسی Rust
ساخت اپ‌های آموزشی: آموزش کاتلین (چندزبانه) و ماشین‌حساب علمی
ادامهٔ برنامهٔ انتشار هفتگی — چالش و تمرین‌های تازه

مسیر کامل ۱۲ درس Rust — یک نگاه

#	عنوان	وضعیت
۱	Rust چیست و چرا؟	✔ منتشر شده
۲	نصب و اولین برنامه	✔ منتشر شده
۳	چاپ متن با println!	✔ منتشر شده
۴	کامنت‌ها و پیام‌های کامپایلر	✔ منتشر شده
۵	متغیرها: let و let mut	✔ منتشر شده
۶	انواع داده: i32 و f64	✔ منتشر شده
۷	رشته‌ها	✔ منتشر شده
۸	تصمیم‌گیری: شرط if	✔ منتشر شده
۹	حلقه‌های loop و while	✔ منتشر شده
۱۰	آرایه‌ها و حلقهٔ for	✔ منتشر شده
۱۱	توابع با return	✔ منتشر شده
۱۲	سازه‌ها (struct)	✔ منتشر شده

پرونده: چرا Rust از همه‌چیز سخت‌گیرتر است؟

در طول این دوازده درس بارها دیدید که کامپایلر Rust خطا می‌دهد؛ حتی وقتی کد «به نظر» درست می‌آید. مثلاً E0384 وقتی می‌خواهید مقدار متغیر قفل‌شده را عوض کنید، یا E0308 وقتی جعبهٔ عددی را با متن پر می‌کنید. چرا Rust این‌قدر سخت می‌گیرد؟ جوابش سه راز دارد.

۱) مالکیت (Ownership) — هر چیز یک مالک دارد

در Rust هر تکه‌داده یک مالک روشن دارد. وقتی مالک تمام شود (مثلاً تابع تمام شود)، حافظه خودبه‌خود آزاد می‌شود — بدون زباله‌روب، بدون نشتی حافظه. برای همین Rust هم سریعِ C است و هم امن.

۲) قرض‌دادن (Borrowing) — یا مالک باش، یا قرض بگیر

می‌شود داده را به تابع «قرض» داد تا فقط بخواند یا با اجازه عوض کند. Rust حساب‌دار دقیقی است: هم‌زمان نمی‌گذارد دو نفر یکی را عوض کنند. نتیجه؟ همان باگ‌هایی که در زبان‌های دیگر وسط شب گریبان برنامه‌نویس را می‌گیرند، این‌جا موقع کامپایل گرفته می‌شوند.

۳) تغییرناپذیری پیش‌فرض — قفل تا وقتی کلید دادی

یادتان هست درس ۵؟ متغیر پیش‌فرض قفل است و تا ننویسید let mut، تغییر نمی‌کند. این سخت‌گیری باعث می‌شود برنامه‌ها قابل پیش‌بینی و مطمئن‌تر باشند.

پیام اصلی برای شما: وقتی کامپایلر Rust خطا می‌دهد، در حال دفاع از شماست. هر error یک باگ است که قبل از رسیدن به دست کاربر گرفته شده. برنامه‌نویس‌های حرفه‌ای به پیام‌های کامپایلر «دوستِ صمیمی» می‌گویند!

```
// سخت‌گیری مفید!
let rekord = 12;
rekord = 15; // ❌ error[E0384]: cannot assign twice to immutable variable
let mut emtiaz = 10;
emtiaz = 15; // ✅ چون اجازهٔ تغییر دادیم
```

درسنامهٔ ویژه: توابع با return

درس ۱۱ برای خیلی از اعضا سخت‌ترین و در عین حال مهم‌ترین درس مسیر بود؛ چون از «نوشتن کد» به «چیدن کد» می‌رسد. این‌جا خلاصهٔ کاربردی‌اش را برای مرور داریم.

صورت مسئله

می‌خواهیم معدل دو نمره را چند بار در برنامه حساب کنیم. اگر هر بار همان تقسیم را بنویسیم، کد شلوغ و خطاخیز می‌شود. راه‌حل: یک تابع، یک بار نوشتن، هر چند بار که لازم باشد صدا زدن.

```
fn moadele(a: i32, b: i32) -> i32 {
    return (a + b) / 2; // بازگشتِ جواب
}

fn main() {
    println!("{}", moadele(10, 20)); // ۱۵
    println!("{}", moadele(18, 14)); // ۱۶
}
```

سه نکتهٔ امتحانی

۱) نوع ورودی‌ها

پارامترها در پرانتز با نوعشان می‌آیند: a: i32 — مثل معرفی‌کردن مهمان با شناسنامه.

۲) نوع خروجی

بعد از -> نوشته می‌شود؛ اگر تابع جوابی نداشته باشد اصلاً نوشته نمی‌شود.

۳) سبک Rustی

در خط آخر می‌شود return و ; را حذف کرد: عبارتِ بدون سمیکالن خودش جواب است.

تمرین این بخش: تابعی به اسم **sade_bakhtar** بنویسید که یک عدد بگیرد و بگوید اول است یا نه. برای اعداد کوچک، همان راه درس‌های قبل کافی است: حلقه از ۲ تا جذرِ عدد و بررسی باقی‌مانده.

چالش برنامه‌نویسی آبان

🛠 چالش: کتابخانهٔ کوچک با struct

با آموخته‌های درس ۱۲، یک سازهٔ **Ketab** بسازید با سه فیلد: esm (نام کتاب)، nevisande (نویسنده) و sale_chap (سال چاپ). سه نمونهٔ کتاب بسازید و در یک حلقهٔ for روی آرایه، همه را با println! قشنگ چاپ کنید.

امتیاز ویژه: تابعی به اسم tozih بنویسید که یک Ketab بگیرد و یک جملهٔ معرفی برگرداند (با return). کدتان را در کانال بفرستید؛ بهترین‌ها در شمارهٔ بعدی معرفی می‌شوند.

بازی کدِ رمزی — کلمه‌ها را پیدا کن!

در جدول زیر، شش واژهٔ این شماره قایم شده: **LET ، PRINTLN ، LOOP ، FN ، STRUCT ، RUST** (افقی، عمودی و مورب). جای خالی‌ها را با حروف رمز پر کن.

س	ت	ک	ر	ت	س
ل	_	_	_	_	_
ف	_	_	_	_	_

پاسخ‌ها را دستی حل کنید و حاصل را در چالش اصلی بگنجانید؛ عکسی از جدولِ حل‌شده هم می‌تواند همراه کدتان به کانال بیاید.

پازل نیم‌خطی

کدام خط از کد زیر خطا می‌دهد و چرا؟ (راهنمایی: درس ۵!)

```
let sen = 15;
sen = sen + 1;
println!("{}", sen);
```

جواب را با کد چالش بفرستید؛ در شمارهٔ بعدی با توضیح کامل می‌آید.

چهره‌ها: آدا لولیس — اولین برنامه‌نویس تاریخ

حدود دو قرن پیش، وقتی هنوز هیچ کامپیوتری وجود نداشت، دختری در انگلستان با ریاضیات بزرگ شد: آدا لولیس (۱۸۱۵–۱۸۵۲)، دختر شاعر مشهور لرد بایرون. مادرش می‌ترسید آدا مثل پدرش شاعر شود؛ پس ریاضی را جدی گرفت. نتیجه؟ آدا ریاضی‌دان شد و شاعرانه‌ترین برنامه‌نویس تاریخ!

آدا با چارلز ببیج روی «موتور تحلیلی» کار کرد؛ ماشینی مکانیکی که قرار بود محاسبات را خودکار انجام دهد. آدا در یادداشت‌هایش الگوریتمی نوشت برای محاسبهٔ اعداد برنولی با همین ماشین — و همین یادداشت امروز به‌عنوان «اولین برنامهٔ منتشرشدهٔ تاریخ» شناخته می‌شود، سال‌ها پیش از آن‌که اولین کامپیوتر الکترونیکی ساخته شود.

اما نکتهٔ قشنگ‌تر: آدا دید آن‌چه ببیج ساخته فقط برای محاسبهٔ عدد نیست؛ یک ماشینِ همه‌کاره است که می‌تواند موسیقی بسازد و نمادها را ترکیب کند. یعنی او «برنامه‌نویسی» را نه به‌عنوان حساب‌کردن، بلکه به‌عنوان خلق‌کردن دید! زبان برنامه‌نویسی «Ada» به افتخار او نام‌گذاری شده و روز «آدا لولیس» هر سال، زنان دانشمند دنیا را گرامی می‌دارد.

اپ‌های تازهٔ انجمن — نصب کن!

اپ	چی دارد؟	برای چه کسی؟
🎓 آموزش کاتلین — نسخهٔ ۲	۵ درس + آزمون، سه زبانه (فارسی/انگلیسی/عربی)، صفحه‌خوانِ دکمه‌ای	شروع‌کننده‌های برنامه‌نویسی
🦀 آموزش Rust	همان ۱۲ درس این نشریه + آزمون هر درس + نشان‌ها	اعضای مسیر Rust
🧮 ماشین‌حساب علمی	نمودار تابع، حل مرحله‌به‌مرحلهٔ معادله، کولاتز، اعداد اول	درس ریاضی و کدنویسی

راه پیش رو و جمع‌بندی

برنامهٔ شماره‌های بعدی

برنامه
اعلام نتایج چالش آبان + معرفی کدهای برگزیده
درس ۱۳ — فهرست‌ها (Vec): آرایه‌ای که بزرگ می‌شود
درس ۱۴ — متدها و impl: رفتار دادن به سازه‌ها
شمارهٔ بعدی نشریه + شروع فصل دوم: پروژهٔ کوچک گروهی

جمع‌بندی آماری این شماره

۱۲ درس

مسیر کامل Rust از صفر تا سازه‌ها منتشر شد

۳ اپ

اپ‌های آموزشی ساخته شد (کاتلین، Rust، ماشین‌حساب)

۱ چالش

چالش struct کتابخانه — با امتیاز ویژه

سه جمله که باید یادت بماند

۱) متغیر پیش‌فرض قفل است؛ تا ننویسی let mut، تغییر نمی‌کند.

۲) تابع یک‌بار نوشته می‌شود و هزار بار صدا زده — با return جوابش را بگیر.

۳) کامپایلر سخت‌گیر، دوستِ توست؛ هر error یک باگ است که قبل از کاربر گرفته شد.

نشریهٔ انجمن کدنویسی

ویژهٔ آبان · پژوهش‌سرای دانش‌آموزی امام هادی (ع)

```
fn entezar() {
let aban = "tamam shod";
let azar = "hanooz dar rah";
(aban, azar) }
```

کانال انجمن کدنویسی: **ble.ir/anjomanfaghihi**

ساخته‌شده با عشق توسط تیم محتوای انجمن · ehsanmorad.ir